PYTHON AUTOMATION FOR BEGINNERS

25 Practical Projects for Files, Spreadsheets, Reports & Everyday Work



About this free sample

PYTHON AUTOMATION FOR BEGINNERS

25 Practical Projects for Files, Spreadsheets, Reports & Everyday Work

This sample shows the cover, the full contents, the Python installation and setup section, and three complete projects (bulk file renaming, duplicate file detection and Excel report generation) with the full source code, sample input, real test output, common errors and safe-use notes, exactly as they appear in the book.

The full book has 136 pages. Page numbers in the contents refer to the full book; sample pages keep their original page numbers.

Code is provided as is, for learning. Run scripts on copies of your files first and keep backups. Each project states what was and was not tested.

Contents

Introduction 6

Chapter 1 Introduction to Python automation 9

Chapter 2 Installing Python and setting up 11

Chapter 3 Python foundations needed 14

Chapter 4 Files, folders and batch renaming 18

Project 1. Bulk file renaming 19
Project 2. Organising a folder by file type 23
Project 3. Duplicate file detection 27
Project 4. Automated file backup with timestamps 31
Project 5. Folder size report 35
Project 6. Image file organisation by date 38

Chapter 5 CSV processing 42

Project 7. CSV cleanup 43
Project 8. Splitting a large CSV 47
Project 9. Simple expense tracking 50
Project 10. Contact list de-duplication 54
Project 11. Data validation report 58
Project 12. Invoice number generator 62

Chapter 6 Excel automation 66

Project 13. Excel report generation 67
Project 14. Merging spreadsheets 71

Chapter 7 PDF and document processing 75

Project 15. PDF text extraction 76
Project 16. Merging PDF files 80
Project 17. Splitting a PDF by page range 83

Chapter 8 Text processing 86
Project 18. Word frequency counter 87
Project 19. Find and replace across files 90
Project 20. Release notes text cleaner 94
Project 21. Log file summary 97

Chapter 9 Reports and spreadsheets 100
Project 22. Monthly report generation 101
Project 23. Email preparation drafts 106

Chapter 10 APIs and responsible web data usage 110
Project 24. API data retrieval 112

Chapter 11 Scheduling 116
Project 25. Scheduled report preparation 119

Chapter 12 Debugging, security and next steps 123

Conclusion 126

Appendix A. Python cheat sheet 127

Appendix B. Common error reference 128

Appendix C. Glossary 130

Appendix D. Project checklist 132

Appendix E. 30-day learning plan 133

Appendix F. References and further reading 135

CHAPTER 2

Installing Python and setting up

Install Python

Download the current stable Python from the official page, https://www.python.org/downloads/, and follow the installer for your system. Python 3.12 or newer is required.

- **Windows:** run the installer. If it offers "Add python.exe to PATH", tick it. The installer also provides the `py` launcher. Check with `py --version` in PowerShell or Command Prompt.
- **macOS:** use the installer from python.org. Check with `python3 --version` in Terminal.
- **Linux:** Python 3 is usually installed. Check with `python3 --version`. On Debian or Ubuntu you may need `sudo apt install python3-venv python3-pip`. This is the general pattern on those systems; I did not test it.

On Windows, type `py` where this book shows `py`. On macOS and Linux, replace `py` with `python3` in every command. Both of these are the same program: `py script.py` on Windows and `python3 script.py` elsewhere.

If your computer has several Pythons (for example one bundled inside another application), check which one runs: `py -0p` on Windows lists the installed ones. Use a normal Python installation, not the copy inside another program.

Project 1. Bulk file renaming

Problem: A folder holds files named IMG_4471.jpg, Beach day.jpg and notes final v2.txt. You want one consistent pattern such as trip_001.jpg.

Expected result: Every file renamed to prefix_NNN.ext in sorted order, lowercase extension, plus an undo log. Without --apply nothing changes.

Required tools: Python standard library only.

Installation: nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

Complete source code (`project01_bulk_rename.py`):

```
"""Project 1: rename the files in one folder to prefix_001.ext,
prefix_002.ext.
Dry run by default; add --apply to really rename."""
import argparse
import csv
from pathlib import Path


def plan_renames(folder, prefix, start):
    # Sort so the numbering is the same every time you run the
script.
    files = sorted(p for p in folder.iterdir()
                   if p.is_file() and p.name !=
"rename_undo_log.csv")
    plan = []
    for number, path in enumerate(files, start=start):
        new_name = f"{prefix}_{number:03d}{path.suffix.lower()}"
        plan.append((path, path.with_name(new_name)))
    return plan


def main(argv=None):
    parser = argparse.ArgumentParser(description="Bulk rename
files.")
    parser.add_argument("folder", type=Path)
    parser.add_argument("--prefix", default="file")
```

```
    parser.add_argument("--start", type=int, default=1)
    parser.add_argument("--apply", action="store_true",
                        help="really rename (default is a dry
run)")
    args = parser.parse_args(argv)

    if not args.folder.is_dir():
        print(f"Error: {args.folder} is not a folder.")
        return 1
    plan = plan_renames(args.folder, args.prefix, args.start)
    # Refuse to start if a target name already belongs to a
different file.
    sources = {src for src, _ in plan}
    for src, dst in plan:
        if dst.exists() and dst not in sources:
            print(f"Stopped: {dst.name} already exists and is not
in the plan.")
            return 1
    if len({dst for _, dst in plan}) != len(plan):
        print("Stopped: two files would get the same new name.")
        return 1

    for src, dst in plan:
        print(f"{src.name}  ->  {dst.name}")
    if not args.apply:
        print(f"Dry run: {len(plan)} files would be renamed. Add
--apply.")
        return 0

    # Two steps (temporary names first) so a swap such as a->b,
b->a is safe.
    temp = []
    for src, dst in plan:
        tmp = src.with_name(src.name + ".renaming")
        src.rename(tmp)
        temp.append((tmp, dst, src))
    for tmp, dst, _ in temp:
        tmp.rename(dst)
    # Undo log: lets you reverse the rename by hand if you change
your mind.
    log = args.folder / "rename_undo_log.csv"
    with log.open("w", newline="", encoding="utf-8") as fh:
        writer = csv.writer(fh)
        writer.writerow(["old_name", "new_name"])
        writer.writerows((s.name, d.name) for _, d, s in temp)
    print(f"Renamed {len(plan)} files. Undo log: {log.name}")
```

```
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Code explanation:

- `plan_renames` builds the whole list of (old, new) pairs before touching anything. That is what makes a dry run possible.
- `sorted(...)` fixes the order, so the numbering is repeatable. `f"{number:03d}"` pads 1 to 001.
- The two checks stop the script if a target name already belongs to another file, or if two files would get the same name.
- Renaming goes through a temporary `.renaming` name first. Without that step, a rename such as a to b while b to a exists would overwrite a file.
- The undo log is a CSV of old and new names, so you can reverse the job by hand.

Sample input: Folder `sample_data/rename_demo` with five files: `Beach day.jpg`, `budget (copy).txt`, `IMG_4471.jpg`, `IMG_4472.jpg`, `notes final v2.txt`.

Sample output (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

Dry run (default):

```
Beach day.jpg  ->  trip_001.jpg
budget (copy).txt  ->  trip_002.txt
IMG_4471.jpg  ->  trip_003.jpg
IMG_4472.jpg  ->  trip_004.jpg
notes final v2.txt  ->  trip_005.txt
Dry run: 5 files would be renamed. Add --apply.
```

Real run:

```
Beach day.jpg  ->  trip_001.jpg
budget (copy).txt  ->  trip_002.txt
IMG_4471.jpg  ->  trip_003.jpg
IMG_4472.jpg  ->  trip_004.jpg
notes final v2.txt  ->  trip_005.txt
Renamed 5 files. Undo log: rename_undo_log.csv
```

How to run it (from the `source` folder):

```
py project01_bulk_rename.py sample_data/rename_demo --prefix trip
py project01_bulk_rename.py sample_data/rename_demo --prefix trip
--apply
```

These commands change or create files. To repeat a test, delete the `sample_data` folder and run `py make_sample_data.py` again.

Common errors and fixes:

- **Stopped: trip_001.jpg already exists and is not in the plan.** A file with a target name is already there. Choose another --prefix or --start.
- **PermissionError.** A file is open in another program. Close it and run again.

Safe-use notes: Run the dry run first and read every line. Keep the undo log. The script renames only files directly in the folder, never sub-folders.

Optional challenge: Add an option --undo that reads rename_undo_log.csv and restores the old names.

Project 3. Duplicate file detection

Problem: Copies of the same file are scattered over a folder tree and waste space. You want a list, not an automatic clean-up.

Expected result: A text report listing groups of files with identical content and the bytes that could be freed. Nothing is deleted or moved.

Required tools: Python standard library only.

Installation: nothing to install. Use `py` on Windows or `python3` on macOS and Linux to run it.

Complete source code (`project03_find_duplicates.py`):

```
"""Project 3: find duplicate files by content (SHA-256). Report
only:
this script never deletes, moves or renames anything."""
import argparse
import hashlib
from collections import defaultdict
from pathlib import Path


def file_hash(path, chunk_size=1024 * 1024):
    digest = hashlib.sha256()
    with path.open("rb") as fh:
        # Read in chunks so a huge file does not fill the memory.
        while chunk := fh.read(chunk_size):
            digest.update(chunk)
    return digest.hexdigest()


def find_duplicates(folder):
    by_size = defaultdict(list)
    for path in folder.rglob("*"):
        if path.is_file():
            by_size[path.stat().st_size].append(path)
    groups = defaultdict(list)
    for size, paths in by_size.items():
        if len(paths) < 2:  # a file with a unique size cannot be
```

```
a duplicate
            continue
        for path in paths:
            try:
                groups[file_hash(path)].append(path)
            except OSError as error:
                print(f"Skipped {path}: {error}")
    return [sorted(g) for g in groups.values() if len(g) > 1]


def main(argv=None):
    parser = argparse.ArgumentParser(description="Report
duplicate files.")
    parser.add_argument("folder", type=Path)
    parser.add_argument("--report", type=Path,
default=Path("duplicates_report.txt"))
    args = parser.parse_args(argv)
    if not args.folder.is_dir():
        print(f"Error: {args.folder} is not a folder.")
        return 1

    groups = find_duplicates(args.folder)
    lines, wasted = [], 0
    for number, group in enumerate(groups, start=1):
        size = group[0].stat().st_size
        wasted += size * (len(group) - 1)
        lines.append(f"Group {number} ({size} bytes each):")
        lines.extend(f"  {p.relative_to(args.folder)}" for p in
group)
    lines.append(f"{len(groups)} duplicate groups, {wasted} bytes
could be "
                 "freed. Nothing was deleted.")
    args.report.write_text("\n".join(lines) + "\n",
encoding="utf-8")
    print("\n".join(lines))
    print(f"Report saved to {args.report}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Code explanation:

- Files are first grouped by size. Two files with different sizes cannot be identical, so most files are never read.
- `file_hash` reads 1 MB at a time and feeds a SHA-256 digest. Identical content gives an identical digest.
- The `:=` walrus operator reads a chunk and tests it in one line; the loop ends when `read` returns empty bytes.
- The script only writes the report file. There is no delete, move or rename call anywhere in it.

Sample input: Folder `sample_data/dupes`: two identical invoice text files in different sub-folders, two identical 2 KB binary files, and two files that look alike but differ.

Sample output (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

```
Group 1 (2048 bytes each):
  c\photo.bin
  photo_again.bin
Group 2 (18 bytes each):
  a\invoice_march.txt
  b\invoice_march_copy.txt
2 duplicate groups, 2066 bytes could be freed. Nothing was
deleted.
Report saved to duplicates_report.txt
```

How to run it (from the `source` folder):

```
py project03_find_duplicates.py sample_data/dupes
```

Common errors and fixes:

- **PermissionError while hashing.** The script prints Skipped and goes on. Run it on a folder you own.
- **It is slow.** Large folders take time because files of equal size are read in full. Try a smaller folder first.

Safe-use notes: A match means identical bytes, but only you can decide which copy to keep. Delete by hand, after looking at the report, or move to the recycle bin yourself.

Optional challenge: Add a --min-size option to ignore files smaller than a given number of bytes.

Project 13. Excel report generation

Problem: You have a sales CSV and want a formatted workbook with a data sheet and a summary sheet.

Expected result: An .xlsx file with a styled header, frozen top row, number formats, a totals row with formulas, and a Summary sheet.

Required tools: Python standard library plus the `openpyxl` package.

Installation: `py -m pip install openpyxl` on Windows (the `py` launcher), `python3 -m pip install openpyxl` on macOS and Linux. Run it inside your virtual environment.

Complete source code (`project13_excel_report.py`):

```
"""Project 13: turn a sales CSV into a formatted Excel workbook
with a data
sheet and a summary sheet. Needs: python -m pip install
openpyxl"""
import argparse
import csv
from collections import defaultdict
from pathlib import Path

from openpyxl import Workbook, load_workbook
from openpyxl.styles import Alignment, Font, PatternFill


def main(argv=None):
    parser = argparse.ArgumentParser(description="CSV to Excel
report.")
    parser.add_argument("input", type=Path)
    parser.add_argument("output", type=Path)
    args = parser.parse_args(argv)
    if not args.input.is_file():
        print(f"Error: {args.input} not found.")
        return 1
    if args.output.exists():
        print(f"Error: {args.output} already exists. Choose a new
name.")
        return 1
```

```
    wb = Workbook()
    data = wb.active
    data.title = "Data"
    data.append(["Date", "Region", "Product", "Units", "Unit
price", "Revenue"])
    with args.input.open(newline="", encoding="utf-8-sig") as fh:
        for n, row in enumerate(csv.DictReader(fh), start=2):
            # Real numbers, not text, so Excel can sum and sort
them.
            data.append([row["date"], row["region"],
row["product"],
                         int(row["units"]),
float(row["unit_price"]),
                         f"=D{n}*E{n}"])  # a formula: Excel
calculates it
    last = data.max_row
    data.append(["", "", "", f"=SUM(D2:D{last})", "",
f"=SUM(F2:F{last})"])

    header_fill = PatternFill("solid", start_color="1F4E78")
    for cell in data[1]:
        cell.font = Font(bold=True, color="FFFFFF")
        cell.fill = header_fill
        cell.alignment = Alignment(horizontal="center")
    for cell in data[data.max_row]:
        cell.font = Font(bold=True)
    for row in data.iter_rows(min_row=2, min_col=5, max_col=6):
        for cell in row:
            cell.number_format = "#,##0.00"
    for letter, width in zip("ABCDEF", (12, 10, 12, 8, 12, 12)):
        data.column_dimensions[letter].width = width
    data.freeze_panes = "A2"

    # Python's own totals go on the summary sheet as plain
numbers.
    totals = defaultdict(float)
    with args.input.open(newline="", encoding="utf-8-sig") as fh:
        for row in csv.DictReader(fh):
            totals[row["region"]] += int(row["units"]) *
float(row["unit_price"])
    summary = wb.create_sheet("Summary")
    summary.append(["Region", "Revenue (calculated by Python)"])
    for region, value in sorted(totals.items()):
        summary.append([region, round(value, 2)])
    summary.column_dimensions["A"].width = 14
```

```
    summary.column_dimensions["B"].width = 30
    for cell in summary[1]:
        cell.font = Font(bold=True)
    wb.save(args.output)
    print(f"Saved {args.output} ({last - 1} data rows)")

    # openpyxl writes formulas but does not calculate them. Prove
it:
    check = load_workbook(args.output, data_only=True)["Data"]
    print("Formula cell F2 as openpyxl reads it back:",
check["F2"].value)
    print("Python total revenue:", round(sum(totals.values()),
2))
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

Code explanation:

- `Workbook()` makes a new file in memory; nothing exists on disk until `wb.save`.
- Units and prices are converted with `int` and `float` so Excel stores numbers, not text.
- `f"=D{n}*E{n}"` writes a formula as text starting with =. Excel calculates it when the file is opened.
- openpyxl does not calculate formulas. The last lines prove it: reading the saved file with `data_only=True` returns None for F2, because no program has calculated and stored a result yet. The Summary sheet therefore holds values calculated by Python.
- The script refuses to overwrite an existing output file.

Sample input: `sample_data/sales/sales_2026_10.csv`, 24 invented sales rows (date, region, product, units, unit_price).

Sample output (real output from my test run, Windows 11, Python 3.14.4, 2026-11-07):

```
Saved sales_october.xlsx (24 data rows)
Formula cell F2 as openpyxl reads it back: None
Python total revenue: 2692.95
```

How to run it (from the `source` folder):

```
py project13_excel_report.py sample_data/sales/sales_2026_10.csv
sales_october.xlsx
```

Common errors and fixes:

- **ModuleNotFoundError: No module named 'openpyxl'.** Install it into the Python you are running: py -m pip install openpyxl.
- **PermissionError on save.** The target workbook is open in Excel. Close it.

Safe-use notes: Never write over the only copy of a workbook. This script refuses if the name exists. openpyxl can lose features such as charts or macros when it edits an existing file; this project only creates new files.

Optional challenge: Add a chart with openpyxl.chart.BarChart showing revenue per region.